

HOW TO BUILD A FAMILY TREE

How to Build a Family Tree

Minerva University

CS110

Prof. Shekhar

December 18, 2024

HOW TO BUILD A FAMILY TREE

Table of Contents

Problem 1	3
Problem 2	4
Problem 3	6
Problem 4	10
Problem 5	11
Local Approach	11
Global Approach	12
Analysis	13
Problem 6	15
Problem 7	17
Class Discussion Reflection	17
LO Reflection	17
HC Reflection	18
AI Statement	18
Appendix	18

It was a pleasure to be in your class this semester. Thank you for contributing to my future success in technical interviews with your excellent teaching skills on these topics!

HOW TO BUILD A FAMILY TREE

Problem 1

How do we even find the LCS? Well, we create a dp table where we break the problem down by comparing letters at a time. For example, this is the dp table for ABC and ABX:

	a	b	x
a	1	1	1
b	1	2	2
c	1	2	2

Figure 0: LCS DP table

In this dp table, we consider the LCS between all order consecutive substrings of abc and abx that start at a. So, for example, we can find the LCS of a and abc by looking at the bottom of the first column, and the LCS of ab and ab by looking at the middle square. Now we can use dynamic programming as we can deconstruct this into overlapping subproblems. The LCS between two strings if their last characters match is the LCS between the two strings without the last character, plus 1. In this way, we could move diagonally down if a character matches. If the character does not match we would have to compare the LCS between the first string without its last character and the second string and the LCS of the first string and the second string without its last character. Whichever is greater, achieves the LCS between the two complete strings so we would either move horizontally or vertically. This has an optimal substructure as we can reconstruct the problem entirely from these different parts.

To reconstruct the LCS we would start at the last entry (bottom right) and move in the direction of the steepest descent, however, if there is no maximum between the two possible LCSs that I mentioned before (moving horizontally or vertically), then we would have to consider them both as we backtrack to produce all possible LCSs. However, if we just wish to get a length, then we would want to avoid backtracking which is why we would have to define these as two separate functions. Hence, to provide a succinct format for representing both of these functions I have introduced a class called LongestCommonSubsequences, with these as methods. Note that since Python is an object-oriented programming language, I prefer (and is optimal) to define these as classes so that I can succinctly call them in later functions, methods, or classes in a more organized manner rather than having a bunch of functions floating around. It also improves code readability.

HOW TO BUILD A FAMILY TREE

Problem 2

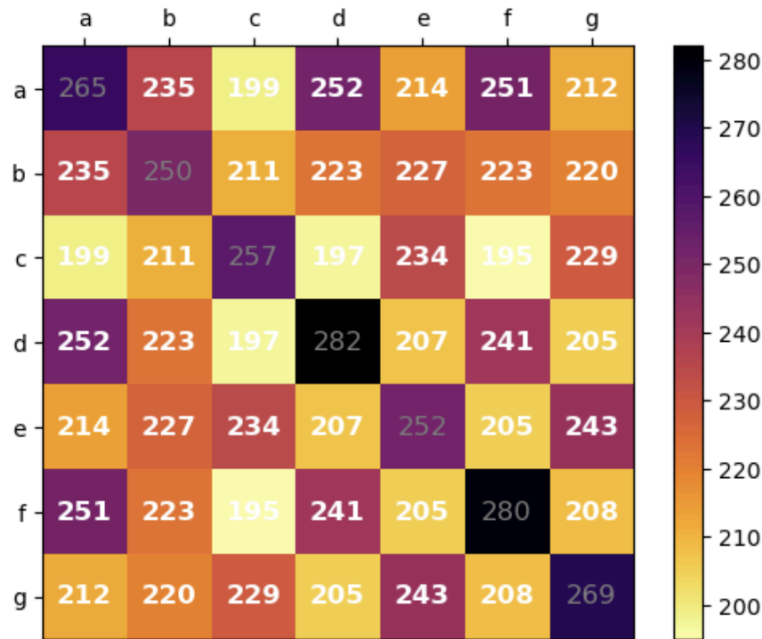


Figure 1: len_lcs_matrix- colored based on the numerical value of LCS of set_strings for visual clarity (Appendix 2)

Since there are 7 strings and the longest common substring is defined to be between two of these, the number of LCSs is the same as the number of combinations you can pick 2 socks out of a sock drawer of 7, which is just 7 choose 2 or $7!/(5!*2!) = 21$ ways. So there are 21 LCS lengths in set_strings, however, in my matrix, I have added the LCS between any two (not necessarily distinct, or novel) strings, to make visualization later in this paper simpler.

I made sure that when you take the i th row and the j th column of my matrix (`len_lcs_matrix[i,j]`), this is the LCS between the i th and j th indexed strings in set_strings (remember indexing starts with 0 though). When creating my len_lcs_matrix I color-themed it so it is easy to see the strongest connections which are the darkest such as [a][d] (252 - dark purple) and [a][f] (251 - dark purple). From this matrix, you can infer which strings are more related to each other. For example, *a* has a greater LCS in common with *d* than it does with *c*, meaning that *a* underwent more insertions/deletions/mutations to get to *c* than it did to get to *d*, which you might be able to infer that *a* hypothetically could be a grandparent, *d* the child, and *c* some grandchild. It is important to take the LCS and not just the subset of letters that two strings have in common, because we want to infer what part of the original string remained unchanged, and generally since insertions/mutations/deletions happen with some probability, a smaller LCS will indicate a further distance as each character the original string that was not changed the first time has another chance of having a change on it thus a smaller LCS.

HOW TO BUILD A FAMILY TREE

It is incredibly difficult to build the resulting genetic tree from this alone. Instead of easily this we can just kind of take note of which strings are probably directly to the others:

a: *b,d,f*
b: *a*
c: *e,g*
d: *a,f*
e: *c,g*
f: *a,d*
g: *e*

Since *b* is not very closely related to anything other than *a* by quite a fair amount I think we can draw a line from *a* to *b*. Since *a* is very closely related to *d* and *f* as well we can set *b* as the root and make *a* have children of *d* and *f*. Similar reasoning we can draw a line from *g* to *e*. Since both *c* and *b* are more closely related to *e* we can make *e* the second child of *b*, and *c* and *g* children of *e*. Let's put the greater LCS relationships on the left for consistency. So our tree will look like this:

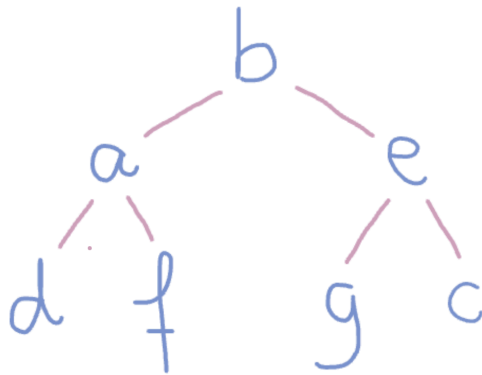


Figure 2: len_lcs_matrix genealogical inference

HOW TO BUILD A FAMILY TREE

Problem 3

Local Approach:

I will use a greedy approach to solve this problem. I will only consider current levels, meaning that when I am analyzing a given node then I will only consider the possible children of this node and the node's relationships with the children, and not the grandchildren, great-grandchildren, and so forth.

My approach is greedy because it takes the best possible move at a given stage in a tree. So for the last complete¹ level of the tree (and only considering the nodes that do not yet have children), a given step in my algorithm will be to assign the children of that node, so the two strings that are mutations of that given string at the node. When I claim that my algorithm is greedy, is that it maximizes the summed LCS between this node string and each of its children, this way my algorithm steps in the way of the steepest ascent towards the objective function of maximizing the total LCS. Note that each step is giving two children as a gene splits into two and mutates, so at each step in the actual mutation of strings it divides in two, and hence my greedy approach follows this same structure.

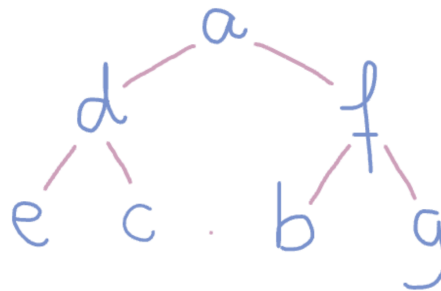


Figure 3: greedy algorithm tree (Appendix 3a)

We can verify that my greedy algorithm works as desired by referring to the `len_lcs_matrix`. We can see that the greatest 2 LCSs between all nodes is at branching out to d and f, and afterward, d has the greater 2 LCSs than f. I provide sufficient tests for my code to verify that it works regardless of the order of the `set_strings` list and that it works for all strings being the same, furthermore, it works for a very simple test case that I have devised. It also works for different numbers of strings.

Global Approach:

For this approach, I deployed a metric called `relatability_percent`. This calculates the probability that two strings are related using their LCS. Now if we have a given string of length 200 and another of length 100, and their LCS is 100, that means that the second string is 100% the child of the first string as it has all of its characters in common. In reality, this is not true, but this is a good enough metric for the purposes of this algorithm. Now the reason why we need to compare the LCS to the strings themselves is because let us say that the LCS is 120, this says nothing about

¹ This ensures that we satisfy the requirement of a perfect binary tree, that its left and right subtrees are balanced at any given node

HOW TO BUILD A FAMILY TREE

the relatability between two strings as we do not know their lengths. If the strings have lengths 125 and 130, this is a really good LCS, however, if they have lengths of 100000 and 200000, this is a pretty poor LCS, so it is relative to the strings themselves. However, I will only compare the LCS to the smallest string as we want to find what percentage of the child's small string is in the parent string.

In my approach, I will wish to maximize the relative LCS between all parents and their respective children in the tree for all possibilities of the tree. This is a global approach as it considers all relationships. However, because these relative LCSs change when the nodes change, it is not possible to represent this tree as one of the trees we learned in class, as this tree focuses more on relative relationships between nodes instead of the values of nodes themselves, so it will need a different structure.

I will use a dynamic programming approach to structure this. To apply dynamic programming we will have to satisfy two conditions, first, we must have overlapping subproblems, and second, we must have optimal substructure (subproblems combined must form the complete solution). We can divide our problem into the overlapping subproblems as so:

maximum relative percent sum tree = maximum(relatability_percent(root, left subtree root) + relatability_percent(root, right subtree root) + maximum relative percent sum right subtree + maximum relative percent sum left subtree)

This iterates through all combinations of left and right subtrees and is more efficient than a brute force method because it will memoize these subtrees (however, is still inefficient, because of the number of combinations however 2^n is better than $n!$). In this case, we can divide our tree into its subtrees and the connections of the root with these sub-trees, and these are overlapping because together they form the sum of all relations in the tree as shown in Figure 4.

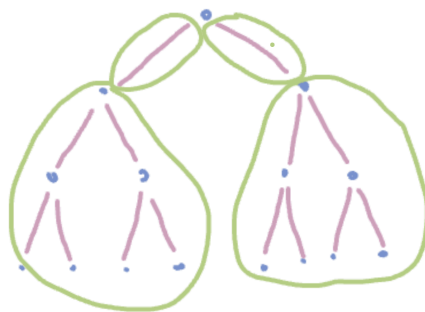


Figure 4: dynamic programming subproblem division

I implemented this by calling on left and right subtree combinations recursively. After implementing my program in Appendix 3b, it resulted in this tree below:

HOW TO BUILD A FAMILY TREE

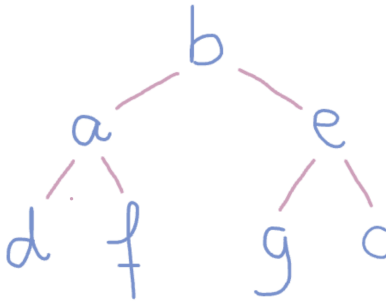


Figure 5: dynamic programming tree (Appendix 3b)

We can see from Appendix 3b that the total sum of relative LCS (reliability_percent) will be 5.639, or in other words the average reliability percent for each edge is $5.639/6 = 0.94$, which is a really high percentage, meaning that we are on the right track. I compared my tree with other trees from my classmates who used different metrics to make sure that my tree was optimal for my algorithm and it is.

Furthermore, I would like to explain why my tree is highly likely to be the original based on my metric, and why my algorithm is functional.

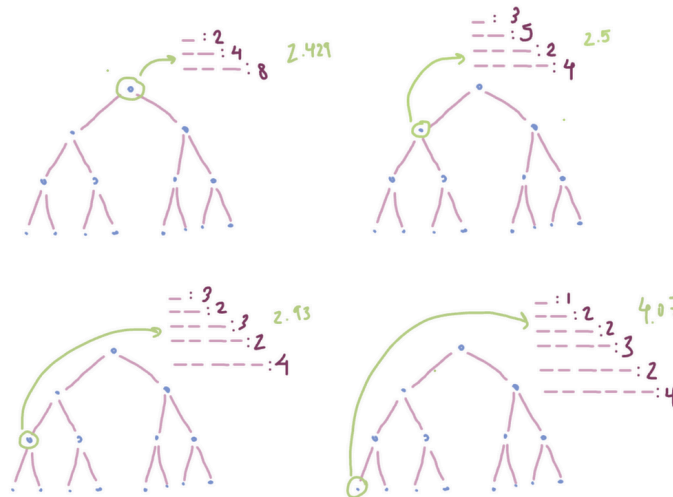


Figure 6: Diagrams measuring the average distance in edges from a node to another node in the tree

As we can see the root is on average the closest to all other nodes, and then it is the second level, and then the third level, and so on. This means that to maximize this tree, we would want the root to have the maximum total LCS relativity compared to all the other nodes, and then the nodes in the second level will have the next greatest, and so forth. Checking with my derived tree this is correct. Meaning that based on my metric this is the global optimal solution, which means it is a successful dynamic programming approach.

Video: <https://drive.google.com/file/d/1wMT56eC7I2WJD5VhXmGnWtBVdqONf8Dg/view?usp=sharing>

HOW TO BUILD A FAMILY TREE

Problem 4

The greedy approach does not find a global optimal to its objective function. The greedy approach only looks at the next step and does not consider future steps of the algorithm, like the relationship of the grandchildren with the children of the root (which there are more of, so should also probably matter more on average!), which is why it will not maximize the tree. It does not consider all relationships at once, so it will neglect some nodes and might not maximize their relationships. For instance, in the optimal subtree, I drew from the `len_lcs_matrix`, it considers how *b*'s strongest connection is *a* by quite a long shot which is why it would make *b* the root, however, the greedy approach will not consider this (hence I do not consider my greedy approach to give me a tree similar to the one from `len_lcs_matrix`). Furthermore, I deployed a different metric for my dynamic tree that considers the relative LCS, which will also cause some differences. Adding relative LCS to my greedy approach would further optimize my greedy approach as it would analyze how strong the relative connection between a parent and the child is and compare those instead of the LCS (which as I discussed in Problem 3 doesn't tell us much about the strength of connection without the length of the strings themselves). However, even with this metric for the reasoning described before, my greedy approach would not guarantee a global optimal, and therefore this is a limitation of that algorithm.

Hence I can conclude that I do not expect my trees to be the same, however, I do expect maybe some sets of parents and children to be alike.

While my algorithm for finding the `len_lcs_matrix` tree from 2c and my global approach sound pretty different on paper, when finding my tree in 2c I considered the relative relationship by maximizing the relationship on average, which is similar to what I have done to my dynamic programming approach. While I was surprised at first to find these two trees identical, now taking a second look, it is not at all that surprising anymore.

HOW TO BUILD A FAMILY TREE

Problem 5

Local Approach

Given 2 strings a and b the time complexity to find their LCS, is the same time complexity as to create a table with length a and length b . Hence if we say that the average gene length is M then the LCS would be $O(M^2)$, however, that being said it is always $O(M^2)$, because the way the dynamic programming approach works is it calculates all the solutions, memoizes them, and then we can just retrieve them, henceforth, we always have the same efficiency, hence we can generalize this to $\Theta(M^2)$ efficiency.

For the actual greedy tree, we will be going through N strings, and to compute the root we compare every string to every other string so our complexity for `find_root` is $\Theta(N^2)$. Now for the `build tree` method, we have 3 main loops, the first is “While True” which goes through all the level nodes, however since the level nodes are always changing it just goes through all the nodes (N), and then we also have 2 for loops within which also goes through the level nodes and the remaining lists, and hence the time complexity is $O(N^3)$. Note that we can just ignore the last while loop as its complexity is irrelevant with respect to the grand scheme of the function. Hence our total time complexity would be $\Theta(M^2 * N^2) + O(M^2 * N^3)$ which in the worst-case scenario produces $O(M^2 * N^3)$, we can set our upper bound or worst-case scenario as $O(M^2 * N^3)$. We can observe these in our empirical plots as we can see that the length of our string grows quadratically when we keep the number of strings constant and scale the string length, showing how our time complexity scales by M^2 as the string length approaches greater values. Similarly, our time complexity scales asymptotically at N^3 as we keep our string length constant and our number of strings increases.

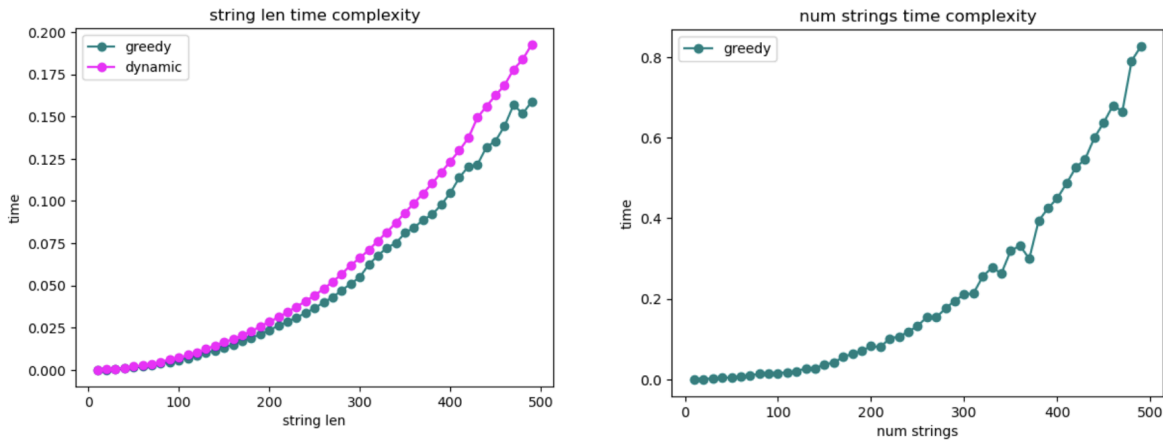


Figure 7: Empirical plots for local approach (Appendix 5)

The memoization within the Greedy Tree class makes sure that we do not calculate the LCS of two strings more than once as we memoize within the class itself. For space complexity, memoization also helps because instead of having to create a new instance in the memory for the LCS of two strings it can just retrieve it from the memoization dictionary. The space complexity for this algorithm is N for the actual `self.tree_list` as it has length $2N+1$, then the space to the LCS computation between every two strings of length M is M^2 as we have a data table of this size for

HOW TO BUILD A FAMILY TREE

each, and since there are n choose two ways to pick two strings it will take up $M^2 \cdot N C_2 + 2N + 1$, however since we only consider higher order scalings when we consider worst case scenario as we take an upper bound this will be $O(M^2 \cdot N^2)$ for space complexity. Note that we disregard lower-order characters as we want to see what happens when N and M grow asymptotically, so lower-order variables are negligible.

Global Approach

Again to find the relatability_percent we will need to construct a dp table with M^2 values in it which will take up M^2 efficiency. We will need to go through all possible sub-tree combinations meaning that a character can either appear in the combination or it might not which leads to a 2^N complexity for this, hence this scales to an $O(2^N \cdot M^2)$ complexity, however it is worth noting that the memoization of the subtrees in my algorithm guarantees that I only consider each subtree once, otherwise we might have been recalculating subtree at each level, making it a potential $O(4^N \cdot M^2)$ time complexity, which is worse. This is why I chose to incorporate memoization into my algorithm. We can confirm that it is multiplication because this is what happens in our algorithm when we vary our constant of M from 3 to 7, the graphs diverge sooner, meaning that the relationship is dependent on M (well M^2).

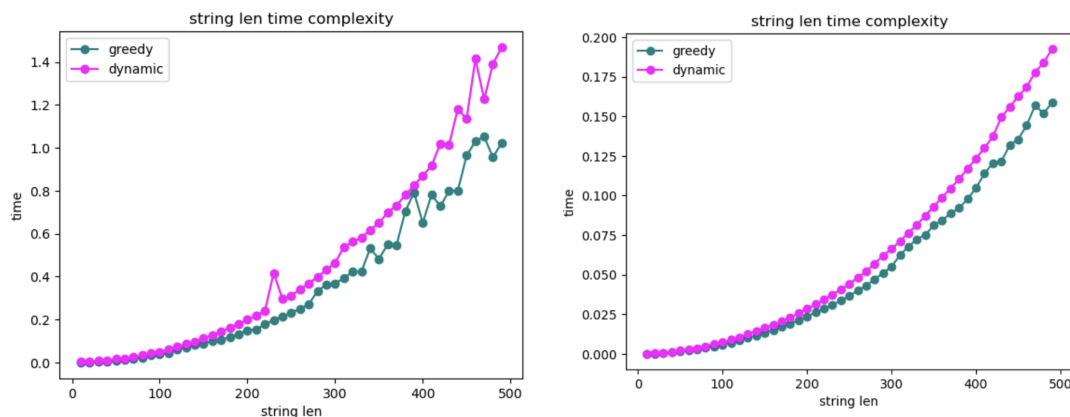


Figure 8: change in string length constant (Appendix 5)

Dynamic programming is very cool because you calculate all of the possibilities exactly once and will not calculate a given sub-problem more than once if done right, and then try to construct a solution from them. However, since I am going through all of the possibilities we can see how it might scale inefficiently to 2^N . Note that we still have the $N^2 \cdot M^2$ term however, this term is added to the 2^N and proves insignificant when we scale both numbers asymptotically so our final time complexity is $\Theta(2^N)$, as a tight bound for our program as we always count all combinations. We can observe this to reflect on my time complexity plots in Figure 9, as we see that the scaling jumps dramatically even when we only increase N by a little.

HOW TO BUILD A FAMILY TREE

The time complexity of my algorithm is not optimal as it scales exponentially, which is why we might consider a greedy algorithm to create more time-efficient programs even if they might not give a globally optimal solution. For space complexity again since we have N strings of length M , then we can see that the space complexity required to store these would be $N^2 * M^2$, however, we would further have to store the combinations as well which would lead to 2^N , which will dominate this as N scales and hence we have $O(2^N)$ asymptotic growth for space complexity too, making the dynamic programming approach less efficient in both time and space complexity by far.

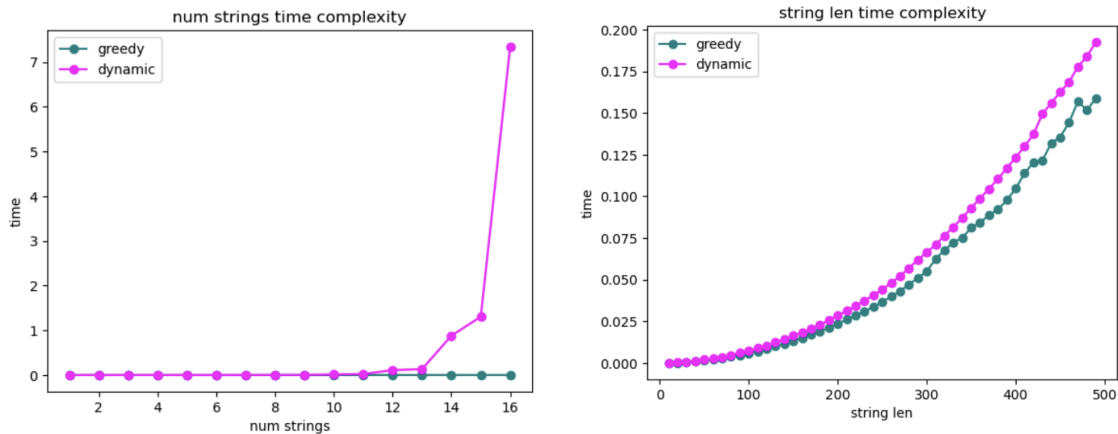


Figure 9: Empirical plots comparing local and global approaches (Appendix 5)

Analysis

Look at Figure 9. As we can see the local and global approaches both have similar asymptotic plot scalings when it relates to the string length, however, the local approach outperforms the global approach significantly when we change the number of strings because of the 2^N factor in the time complexity for the dynamic approach (compared to the N^3 factor in the local approach). Hence we can conclude the local approach to be more efficient with time complexity, and using similar reasoning we can conclude it has more efficient space complexity as well.

My dynamic approach is not efficient, which I am not proud of given the amount of time I spent thinking and writing the code for my global approach. However, I do believe that there is nothing else I can do to fix this time complexity given that a dynamic approach must go through all possibilities of a tree, in my case, I considered all combinations, and I made sure to improve the time and space complexities (even by a little) by incorporating memoization, but since I must reconstruct the tree, I do not believe there is any way to mitigate this high scaling of time complexities. That being said, my dynamic approach does scale better than a brute force approach as for a brute force you would just consider all permutations of a tree and that has a time complexity of $M^2 * N!$ (we still need to consider relatability percent which is why we have the M^2 term), which will scale worse than my algorithm as $N!$ is greater than 2^N as N grows.

$$2^5 = 32$$

$$5! = 120$$

HOW TO BUILD A FAMILY TREE

$$2^{10} = 1,024$$

$$10! = 3,628,800$$

$$2^{20} = 1,048,576$$

$$20! = 2,432,902,008,176,640,000$$

This is why I would argue that even though my scaling for my global approach is not optimal, it is better than a brute force approach and hence I would call my dynamic programming effective.

HOW TO BUILD A FAMILY TREE

Problem 6

Given a family tree with genes, there are two possible approaches I can take. The first approach is comparing parents to children and writing a program that finds exactly the number of insertions, deletions, and mutations it might take to get from the parent to the child. The second approach is could write a bunch of equations relating insertions, deletions, and mutation probabilities with the lengths of the parents, children, and their common LCS. I will do the second approach. As for me, I am more curious about how these probabilities relate to numbers and each other. I also believe that this approach is easier, making it more efficient for me to evaluate.

Let the probability that a given character is inserted, deleted, mutated, or done nothing be i, d, m , and n respectively. Since these are the only possible operations done to a character, these must total to 1. Therefore we have equation 1.

$$\text{Eq1: } i + d + m + n = 1$$

Now let us say the length of the Parent is par , the child is $chld$, and their LCS length is lcs . The number of characters in the parent must be the number of characters in the parent plus the number of newly inserted nodes minus the number of nodes that were deleted, which we can then form equation 2 with. (Note that mutations and doing nothing to a character do not change the number so we do not consider them)

$$\text{Eq2: } par + i*par - d*par = chld$$

When you insert a character the character that you perform the insertion on stays the same, similarly when you do nothing to a character that character stays the same. However, when you perform a mutation you change the character, and when you delete a character you remove that character so it does not stay the same. Henceforth we can say that the number of characters that stays the same (the length of the LCS) is the number of insertions plus the number of deletions. Hence we get equation 3.

$$\text{Eq3: } par*n + par*i = lcs$$

Furthermore, I would like to argue that a mutation is just the joint probability of an insertion and a deletion happening. For instance given the sequence AB, If I insert C after A and then delete B, we get AC. If I instead mutate B to C, from the start I still get AC. So a mutation is the same as inserting and then deleting a node or $i*n$. However, since mutation takes up 1 character but mutation and deletion take up 2, to provide a more accurate estimate I will multiply by 2. Therefore our fourth equation is:

$$\text{Eq4: } m = 2*(i*n)$$

HOW TO BUILD A FAMILY TREE

Now for every given parent and child in our tree, we can perform this algorithm and calculate the given probabilities by solving these equations, and then find their average. After running this through my algorithm (Appendix 6) that solves these equations we find the probabilities to be: i : 0.0966, m : 0.0096, d : 0.0506, n : 0.8432

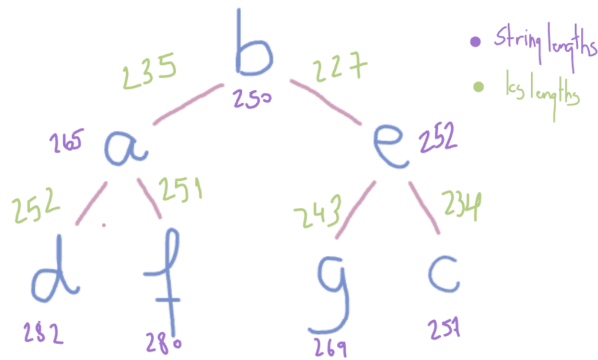


Figure 10: LCS and String Lengths Dynamic Tree

Now we can see if this makes sense, as we can see the children tend to have a larger size than the parents, therefore there should be more insertions than deletions, and therefore the probability of insertion should be greater than deletion which is true. So I would say this is reasonable. Furthermore, I would expect that doing nothing to a node would remain with the greatest probability as the LCS is so big, and there is not much change in character lengths meaning that unless there are a great number of deletions (which it cannot be due to the large size of the LCS) the number of doing nothing to a node should be fairly high, I would estimate this to be somewhere in between 70-90 percent and looking at the strings themselves and comparing them I would also conclude the number of unchanged characters to be this much. This is true based on the result of my algorithm. Hence I would say that my result is reasonable.

Problem 7

Class Discussion Reflection

A class discussion that helped me with my problem-solving was in Session 22 (13.1) when Prof Shekhar, Autinn, and I were debating what was the right way to deconstruct the coin problem (time stamp around 1:30:00). This helped me with my paper as this discussion helped clarify how to break down the coin problem into overlapping subproblems which I was priorly confused about. Thus it inspired me to construct my solution for my dynamic approach.

LO Reflection

#professionalism: I stayed within the specified video length and word count. I used proper grammar throughout my assignment. I maintained a professional tone, and I made sure to provide headers for both my code and text, with a table of contents. (40 words)

#cs110-AlgoStratDataStruct: In video, I explained why my algorithm is greedy (1:30), and without jargon how it works (0:00-1:30). I make sure to specify that dynamic meets its requirements (overlapping-subproblems+optimal-substructure) and how to build bottom-up. I explain how LCS dp table works (Problem1) and memoization structure helps efficiency (Problem5). (50 words)

#cs110-CodeReadability:² I use class objects to improve organization through grouping similar components. I added doc-strings and comments to all my methods and classes (Feedback from Project 1). I excelled in this HC as I made sure my output looks pretty, my matrix is colored making it more readable. I included headers. (Words: 49)

#cs110-ComplexityAnalysis: I explain theoretically how to derive the time complexities and space complexities using a step-by-step explanation. I added several experimental plots. I differentiate between average and worst-case scenarios with asymptotic bounds. I explained how dynamic-approach considers all combinations which is why it's a tight bound for 2^N scaling. (Words: 50)

#cs110-ComputationalCritique: I explain why I chose to use a dynamic instead brute-force since it has less asymptotic time-complexity, however dynamic is inefficient since it considers all possibilities exactly once. I explain why my greedy approach is more efficient by comparing time-complexity and space-complexity of both variables (Problem-Set I didn't analyze space-complexity). (Words: 50)

² Note about code PDF formatting: I made sure all lines of actual code are less than 80 lines so they are readable, however I did not do this for the comments as there are so many comments and if I would have done this it would not have made it very easy to read. Instead if you are more curious about my comments and docstrings then you can go to the ipynb file that I have attached.

HOW TO BUILD A FAMILY TREE

#cs110-PythonProgramming: I wrote a creative solution to each problem. I provided lots of test cases by considering your feedback from Problem-Set, and explained how test cases are sufficient. I also made sure to use assert statements when necessary by changing the order of my input. (Words: 44)

HC Reflection

#levelsofanalysis: To maximize system-level tree relationships we analyze different levels. Individual root and its LCS with other strings. Greedy-approach maximizing the relationship of parent and children (group-level-1). Dynamic-Approach maximizing sub-trees and relationships (group-level-2) with root. To optimize system-level tree, maximizing individuals/subparts isn't enough and must consider the overall relationship of subparts combined. Which is why dynamic is global maximum as it considers subpart relationships with each other, but greedy doesn't. (Words: 70)

AI Statement

AI Statement: *I did not use any AI to write my code, brainstorm, or draft this assignment. However, I used Grammarly to check my sentence-level spelling.*

Appendix

Formating

In [2]: `!pip install "black[jupyter]"`

```
Requirement already satisfied: black[jupyter] in /opt/anaconda3/lib/python
3.12/site-packages (21.12b0)
Requirement already satisfied: click>=7.1.2 in /opt/anaconda3/lib/python3.
12/site-packages (from black[jupyter]) (8.1.7)
Requirement already satisfied: platformdirs>=2 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from black[jupyter]) (3.10.0)
Requirement already satisfied: tomli<2.0.0,>=0.2.6 in /opt/anaconda3/lib/p
ython3.12/site-packages (from black[jupyter]) (1.2.3)
Requirement already satisfied: pathspec<1,>=0.9.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from black[jupyter]) (0.10.3)
Requirement already satisfied: typing-extensions>=3.10.0.0 in /opt/anacond
a3/lib/python3.12/site-packages (from black[jupyter]) (4.11.0)
Requirement already satisfied: mypy-extensions>=0.4.3 in /opt/anaconda3/li
b/python3.12/site-packages (from black[jupyter]) (1.0.0)
Requirement already satisfied: ipython>=7.8.0 in /opt/anaconda3/lib/python
3.12/site-packages (from black[jupyter]) (8.25.0)
Requirement already satisfied: tokenize-rt>=3.2.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from black[jupyter]) (6.1.0)
Requirement already satisfied: decorator in /opt/anaconda3/lib/python3.12/
site-packages (from ipython>=7.8.0->black[jupyter]) (5.1.1)
Requirement already satisfied: jedi>=0.16 in /opt/anaconda3/lib/python3.1
2/site-packages (from ipython>=7.8.0->black[jupyter]) (0.18.1)
Requirement already satisfied: matplotlib-inline in /opt/anaconda3/lib/pyt
hon3.12/site-packages (from ipython>=7.8.0->black[jupyter]) (0.1.6)
Requirement already satisfied: prompt-toolkit<3.1.0,>=3.0.41 in /opt/anaco
nda3/lib/python3.12/site-packages (from ipython>=7.8.0->black[jupyter])
(3.0.43)
Requirement already satisfied: pygments>=2.4.0 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from ipython>=7.8.0->black[jupyter]) (2.15.1)
Requirement already satisfied: stack-data in /opt/anaconda3/lib/python3.1
2/site-packages (from ipython>=7.8.0->black[jupyter]) (0.2.0)
Requirement already satisfied: traitlets>=5.13.0 in /opt/anaconda3/lib/pyt
hon3.12/site-packages (from ipython>=7.8.0->black[jupyter]) (5.14.3)
Requirement already satisfied: pexpect>4.3 in /opt/anaconda3/lib/python3.1
2/site-packages (from ipython>=7.8.0->black[jupyter]) (4.8.0)
Requirement already satisfied: parso<0.9.0,>=0.8.0 in /opt/anaconda3/lib/p
ython3.12/site-packages (from jedi>=0.16->ipython>=7.8.0->black[jupyter])
(0.8.3)
Requirement already satisfied: ptyprocess>=0.5 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from pexpect>4.3->ipython>=7.8.0->black[jupyter]) (0.
7.0)
Requirement already satisfied: wcwidth in /opt/anaconda3/lib/python3.12/si
te-packages (from prompt-toolkit<3.1.0,>=3.0.41->ipython>=7.8.0->black[jup
yter]) (0.2.5)
Requirement already satisfied: executing in /opt/anaconda3/lib/python3.12/
site-packages (from stack-data->ipython>=7.8.0->black[jupyter]) (0.8.3)
Requirement already satisfied: asttokens in /opt/anaconda3/lib/python3.12/
site-packages (from stack-data->ipython>=7.8.0->black[jupyter]) (2.0.5)
Requirement already satisfied: pure-eval in /opt/anaconda3/lib/python3.12/
site-packages (from stack-data->ipython>=7.8.0->black[jupyter]) (0.2.2)
Requirement already satisfied: six in /opt/anaconda3/lib/python3.12/site-p
ackages (from asttokens->stack-data->ipython>=7.8.0->black[jupyter]) (1.1
6.0)
```

```
In [3]: %load_ext blackcellmagic
```

Appendix 1

```
In [5]: """
Installing necessary packages
"""

import time
import numpy as np
import matplotlib.pyplot as plt
import random
from itertools import combinations
```

```
In [6]: class LongestCommonSubsequences:
    """
    A class that creates a data table that finds the length of the longest
    and backtracks to show all of them. LCS is calculated using backtracking.

    Parameters:
    -----
        a : string
            the first strings
        b : string
            the second string

    Returns:
    -----
        Longest Common Subsequence Length
        All Longest Common Subsequences

    """

    def __init__(self, a, b):
        self.a = a
        self.b = b
        self.lena = len(a) # neatly stores the length of a
        self.lenb = len(b) # neatly stores the length of b
        self.lcs_table = [
            [0] * (self.lenb + 1) for i in range(self.lena + 1)
        ] # initializes a dynamic programming table
        self.memoize = {} # memoizes past smaller strings for backtracking
        self.fill_lcs_table() # populates the table when the object is created

    def fill_lcs_table(self):
        """
        Fills the lcs_table by building up recursively from the top left to the bottom right.

        How it works
        -----
        It works as if the letter is common then the lcs_table of length i and j is the lcs of
        then it is just 1 plus i-1 and j-1, and if it is not it is the max of the two.

        Parameters
        -----
        None
    """
```

```

Returns
-----

None
"""
for i in range(1, self.lena + 1):
    for j in range(1, self.lenb + 1):
        if (
            self.a[i - 1] == self.b[j - 1]
        ): # if the characters are equal then we have that the l
            self.lcs_table[i][j] = self.lcs_table[i - 1][j - 1] +
        else: # if the characters are not equal it is the maximu
            self.lcs_table[i][j] = max(
                self.lcs_table[i - 1][j], self.lcs_table[i][j - 1]
            )

def get_all_lcs(self):
    """
    Gets all of the common substrings of a and b that have the greatest length.

    Parameters
    -----

    None

    Returns
    -----

    List: list of longest common substrings
    """
    if self.lcs_table[self.lena][self.lenb] == 0:
        return None
    return self.backtrack(
        self.lena, self.lenb
    ) # backtracks from the bottom right (top down)

def get_lcs_len(self):
    """
    Gets the length of the LCS by extracting the bottom right square of the dp table.

    Parameters
    -----

    None

    Returns
    -----

    Int: the length of the longest common subsequence of ordered letters
    """
    return self.lcs_table[self.lena][
        self.lenb
    ] # gets the bottom right square of the dp table

def backtrack(self, i, j):
    """
    Gets ALL of the longest common subsequences, by using recursive to find all.

    Parameters
    -----

    i: the length of the subset of a you want to backtrack up to
    j: the length of the subset of b you want to backtrack up to

    Returns
    -----

```

```

Int: the length of the longest common subsequence of ordered letters
"""
if (i, j) in self.memoize:
    return self.memoize[
        (i, j)
    ] # if the LCSs of i and j have been computing we store them

lcs_set = set()
if (
    i == 0 or j == 0
): # sets up a case, so if we have no lengths of strings then the
    return {""}

if (
    self.a[i - 1] == self.b[j - 1]
): # if the characters match then we add that character to the LCS
    lcs_set = self.backtrack(i - 1, j - 1)
    lcs_set = {
        lcs + self.a[i - 1] for lcs in lcs_set
    } # list comprehension is used to improve efficiency (and loop)
else: # move either up and/or to the left
    if self.lcs_table[i - 1][j] == self.lcs_table[i][j]:
        lcs_set.update(self.backtrack(i - 1, j))
    if (
        self.lcs_table[i][j - 1] == self.lcs_table[i][j]
    ): # notice how this is NOT elif to account for multiple LCS
        lcs_set.update(self.backtrack(i, j - 1))

self.memoize[(i, j)] = lcs_set
return lcs_set # return the list of all longest common subsequences

def display_dp_lcs(self):
    """
    displays the dp lcs table
    """
    for row in self.lcs_table:
        print(row)

```

tests

```

In [8]: test_cases = [
    ( ("ABCBDBAB", "BDCABA"), ["BDAB", "BCBA", "BCAB"], 4),
    ( ("abc", ""), None, 0),
    ( ("abc", "a"), ["a"], 1),
    ( ("abc", "ac"), ["ac"], 2),
    ( ("abcdefg", "gfedcba"), ["a", "b", "c", "d", "e", "f", "g"], 1),
    (
        ("abcdefghijklmnopqrstuvwxyz", "abcdefghijklmnopqrstuvwxyz"),
        ["abcdefghijklmnopqrstuvwxyz"],
        26,
    ),
    ( ("AAAAAAAA", "AAAAAAAA"), ["AAAAAAAA"], 8),
]

for (x, y), expected_lcs, expected_len in test_cases:
    lcs = LongestCommonSubsequences(x, y)
    all_lcs = lcs.get_all_lcs()
    if all_lcs == None or expected_lcs == None:
        assert all_lcs == expected_lcs

```

```

else:
    assert sorted(all_lcs) == sorted(expected_lcs)
assert lcs.get_lcs_len() == expected_len

```

```

In [9]: a = "ABC"
        b = "ABX"

        lcs = LongestCommonSubsequences(a, b)
        lcs.display_dp_lcs()

```

```

[0, 0, 0, 0]
[0, 1, 1, 1]
[0, 1, 2, 2]
[0, 1, 2, 2]

```

Appendix 2

```

In [11]: class LCSMatrix:
        """
        A class that stores the LCSs of a dictionary of strings into a matrix

        Parameters:
        -----
            set_strings : dictionary
                dictionary with name of string and string you wish to compute

        Returns:
        -----
            Longest Common Subsequence Length (based on LCS class)
            Matrix with the lcs of the ith and jth strings in set_strings
        """

        def __init__(self, set_strings):
            self.set_strings = set_strings
            self.memoization = {}

        def get_lcs_length(self, i, j):
            """
            Gets the length of the LCS between strings i and j by extracting

            Parameters
            -----
            None

            Returns
            -----
            Int: the length of the longest common subsequence of strings indexed by i and j

            """
            if (i, j) in self.memoization:
                return self.memoization[
                    (i, j)
                ] # memoizes so that it does not recompute it for example if

            LCSij = LongestCommonSubsequences(
                self.set_strings[i][1], self.set_strings[j][1]
            ) # finds LongestCommonSubsequences between the two strings
            lcs_len = LCSij.get_lcs_len() # gets the length
            self.memoization[(i, j)] = lcs_len # memoizes it
            self.memoization[(j, i)] = lcs_len # memoizes it for the inverse

```

```

        return lcs_len # returns the length

def len_lcs_matrix(self):
    """
    creates

    Parameters
    -----
    None

    Returns
    -----
    matrix: table
        where the LCS between strings indexed i and j is displayed
    """
    len_lcs_matrix = [
        [0] * len(self.set_strings)
        for column in range(len(self.set_strings))
    ] # initializes an empty matrix to fill in

    for i in range(len(self.set_strings)):
        for j in range(len(self.set_strings)):
            len_lcs_matrix[i][j] = self.get_lcs_length(
                i, j
            ) # fills the ith rown and the jth column with the lcs b

    return len_lcs_matrix # returns the matrix

def show_matrix(self):
    """
    Prints matrix nicely to see better stronger relationships with co

    Parameters
    -----
    None

    Returns
    -----
    None
    """
    len_lcs_matrix = self.len_lcs_matrix() # calcs the len_lcs_matrix

    alphabet = [
        "a",
        "b",
        "c",
        "d",
        "e",
        "f",
        "g",
        "h",
        "i",
        "j",
        "k",
        "l",
        "m",
        "n",
        "o",
        "p",
        "q",
    ]

```

```

        "r",
        "s",
        "t",
        "u",
        "v",
        "w",
        "x",
        "y",
        "z",
    ] # alphabet for names
    labels = alphabet[
        0 : len(len_lcs_matrix)
    ] # generates the labels for the axes depending on how many strings

plt.imshow(
    len_lcs_matrix, cmap="inferno_r", interpolation="none"
) # plots the matrix using matplotlib lib with the inferno colors

for i in range(len(len_lcs_matrix)):
    for j in range(len(len_lcs_matrix)):
        if (
            i == j
        ): # if the strings are the same it makes the text less visible
            plt.text(
                j,
                i,
                f"{len_lcs_matrix[i][j]}",
                ha="center",
                va="center",
                color="gray",
                fontsize=12,
            ) # centers the text (I asked the internet how to do this)
        else:
            plt.text(
                j,
                i,
                f"{len_lcs_matrix[i][j]}",
                ha="center",
                va="center",
                color="white",
                fontsize=12,
                fontweight="bold",
            )

plt.xticks(
    ticks=np.arange(len(len_lcs_matrix)), labels=labels
) # marks the place where to put the x labels
plt.yticks(ticks=np.arange(len(len_lcs_matrix)), labels=labels)

plt.gca().xaxis.set_ticks_position(
    "top"
) # positions the labels at the top instead of the bottom for visibility

plt.colorbar() # adds the color bar to understand which number belongs to which color
plt.show() # plots

```

```

In [12]: set_strings = [
    ('a', 'ATGGTGC GAAAGCATCTCTTTTCGTGGCGTGATAAGTTTTATGGTATCCCCGGACGTTGGCTAC'),
    ('b', 'TGGTGC GAAAGCATCTCTTTTCGTGGCGTATAGTTTTATGGTATCCCCGGAACGCTGGCTAC'),
    ('c', 'TCTGTGC GATATACATCTCTATCGTTGCGGTATGTTTTATGTGCATCACCCACGCGCTGGCTAC')
]

```

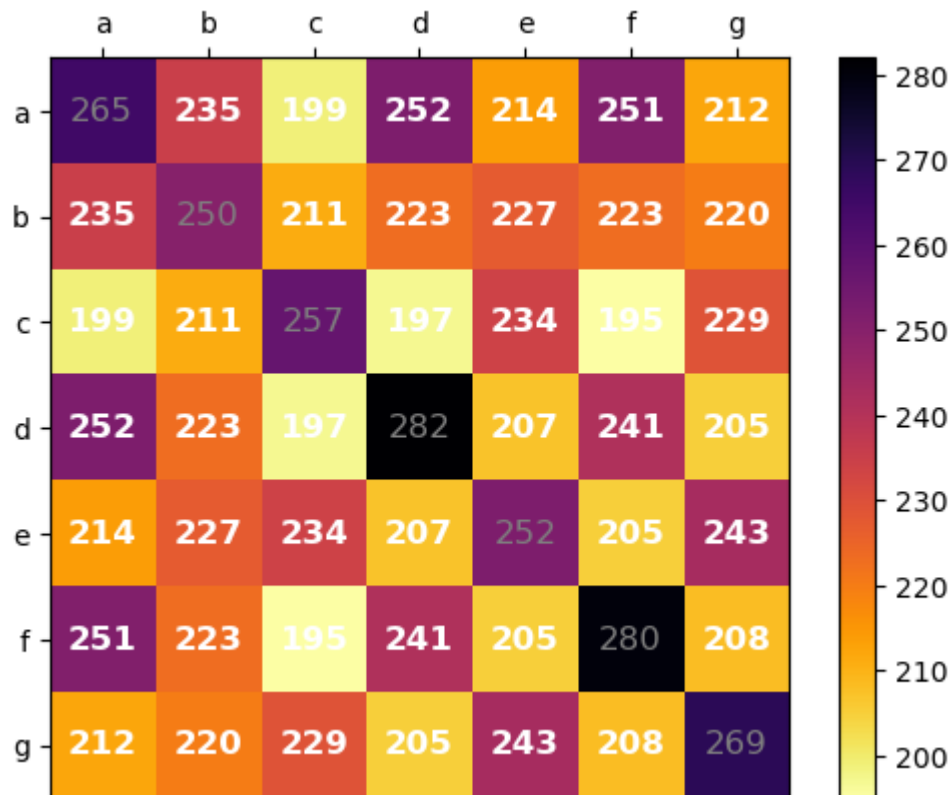


```

('d', 'ATGAGGCGCAAAATTCTCTTTCTCGTGGCGCTGATTAAGTTTTATGTATCCCCGGACGTTGGC')
('e', 'TGGTGCATATACATCTCTTTTCGTGCGTATGTTTTATGGTGATCACCCGGAACCGCTGGCTA')
('f', 'GGGGGAAAGCGATCCCTTATCGTGGCTGTGATAAGTTTTATCGGGTATCCGCCGGACGTTGG')
('g', 'TGGTGCATATACATCTCTTTTCGTGCGTATGTTTTAGGTACACCGGATACGCCTGGCTTAC')
]

set_strings_matrix = LCSMatrix(set_strings)
set_strings_matrix.show_matrix()

```



Appendix 3a

```

In [14]: class GreedyTree:
    """
    A class that constructs a tree storred in an array (children of i are
    The algorithm functions on a Greedy approach by taking the string with
    and then for the last complete level of the tree (grandparent, parent

    Parameters:
    -----
        set_strings : dictionary
            dictionary with name of string and string you wish to compute

    Returns:
    -----
        Best root of the tree
        tree in an array form

    """

    def __init__(self, set_strings):
        self.set_strings = set_strings
        self.tree_list = [None] * (
            2 * len(set_strings) + 1

```

```

    ) # starts a list of length 2 times the length of strings plus 1
    self.added_nodes = (
        set()
    ) # creates a set to store the already added nodes so that we do

def find_root(self):
    """
    Finds the best root of the tree, to start the iterative process. It
    iterates through all the strings for potential roots and
    calculates the longest common subsequence (LCS) between each
    potential root and its children. The root with the highest
    total LCS length is returned.

    Parameters
    -----
    None

    Returns
    -----
    best_root: tuple
        The best root

    """
    total_max = -1 # defines the total max of two kids of a given st.
    best_root = 0 # defines the best root
    lcs_memoize = (
        {}
    ) # if the lcs of two strings has been computed it memoizes it so

    for i in range(
        len(self.set_strings)
    ): # iterates through all the strings for potential roots
        max1, max2 = (
            -1,
            -1,
        ) # defines two maxes for the lcses between a potential root
        for j in range(
            len(self.set_strings)
        ): # iterates through all the strings for potential children
            if i != j: # if the child is not equal to the root process
                lcs_len = lcs_memoize.get((i, j)) # memoize
                if (
                    lcs_len is None
                ): # if it is not in the memoization then it calculates
                    lcs_len = LongestCommonSubsequences(
                        self.set_strings[i][1], self.set_strings[j][1]
                    ).get_lcs_len()
                    lcs_memoize[(i, j)] = lcs_len
                    lcs_memoize[(j, i)] = lcs_len

                if lcs_len > max1: # if this lcs is greater than the
                    max2 = max1 # it sets the old biggest max to the
                    max1 = lcs_len # and sets this to be the biggest
                elif (
                    lcs_len > max2
                ): # else if this lcs is bigger than the second biggest
                    max2 = lcs_len # it sets this second max to the

            if max1 + max2 > total_max: # calculates total max by summing
                total_max = max1 + max2 # updates max
                best_root = i

    return best_root

def build_tree(self):

```

```

"""
Builds the best tree following the greedy method outlines in the c
and then storing it in a list (2i +1 and 2i + 2 are the children c
and then on the last complete level of the tree will find the bes

Parameters
-----
None

Returns
-----
self.tree_list: list
    the tree in the form of a list
"""
root_index = self.find_root() # sets the root index to the best
self.tree_list[0] = self.set_strings[root_index][
    0
] # sets the first item in the tree list to the root index
self.added_nodes.add(
    root_index
) # adds the root to the added nodes list so that we do use the

lcs_memoize = {} # localizes the memoization

while True: # cycles through all the level nodes untill complete
    candidates = [] # candidates for the children
    lvl_nodes = [] # the nodes at the current level

    for i in range(
        len(self.tree_list)
    ): # for i in the range of the tree list
        if (
            self.tree_list[i] is None
        ): # if it reaches a none it breaks the loop as any stri
            break
        elif self.tree_list[
            i
        ] is not None and ( # else if it is not None AND it does
            2 * i + 1 >= len(self.tree_list)
        or (
            self.tree_list[2 * i + 1] is None
            and self.tree_list[2 * i + 2] is None
        )
        ):
            lvl_nodes.append(
                (i, self.tree_list[i])
            ) # then we can add the string in the tree to the le

    if not lvl_nodes: # if it is not in level nodes then it brea
        break

    total_max = -1 # sets the total max
    best_children = (
        {}
    ) # sets a dictionary for the best possible children

    for (
        i,
        node_label,
    ) in (

```

```

        lvl_nodes
    ): # iterates through some index of a label in the tree_list
        node_idx = self.find_index(
            node_label
        ) # finds the index in the original set_strings
        max1 = (
            -1
        ) # sets the two maximum LCS between some node and its p
        max2 = -1
        l_chld = (
            None # defines the left and right children of a give
        )
        r_chld = None

        for chld_idx in range(
            len(self.set_strings)
        ): # for some potential child index in the range of set_
            if (
                chld_idx in self.added_nodes
            ): # if the child is already in added nodes then it
                continue

            lcs_len = lcs_memoize.get(
                (node_idx, chld_idx)
            ) # otherwise it follows a similar logic to root sel
            if lcs_len is None:
                lcs_len = LongestCommonSubsequences(
                    self.set_strings[node_idx][1],
                    self.set_strings[chld_idx][1],
                ).get_lcs_len()
                lcs_memoize[(node_idx, chld_idx)] = lcs_len
                lcs_memoize[(chld_idx, node_idx)] = lcs_len

                if lcs_len > max1:
                    max2 = max1
                    max1 = lcs_len
                    r_chld = l_chld
                    l_chld = chld_idx
                elif lcs_len > max2:
                    max2 = lcs_len
                    r_chld = chld_idx

            if max1 + max2 > total_max:
                total_max = max1 + max2
                best_children = {
                    "parent": i,
                    "l_chld": l_chld,
                    "r_chld": r_chld,
                }

        if not best_children:
            break

        par_idx = best_children[
            "parent"
        ] # extracts the over all best parent and best left and righ
        l_chld = best_children["l_chld"]
        r_chld = best_children["r_chld"]

        while 2 * par_idx + 2 >= len(

```

```

        self.tree_list
    ): # makes sure list is long enough
        self.tree_list.extend([None] * len(self.tree_list))

    if (
        l_chld is not None and l_chld not in self.added_nodes
    ): # adds the left child to the tree based on the dynamic s
        self.tree_list[2 * par_idx + 1] = self.set_strings[l_chld
        self.added_nodes.add(l_chld)

    if (
        r_chld is not None and r_chld not in self.added_nodes
    ): # adds the right child to the tree based on the dynamic s
        self.tree_list[2 * par_idx + 2] = self.set_strings[r_chld
        self.added_nodes.add(r_chld)

    return self.tree_list

def find_index(self, node_label):
    """
    From some label of the dictionary it returns the index in the origi

    Parameters
    -----
    node_label: string
        the label of the string so for example "a", or "b"

    Returns
    -----
    i: int
        index of the label of the string
    """
    for i, (label, string) in enumerate(
        self.set_strings
    ): # iterates through set_strings and checks to find the index o
        if label == node_label:
            return i # if the label and the node label are equal ret
    return None

```

```

In [15]: set_strings = [
    ('a', 'ATGGTGCGAAAGCATCTCTTTTCGTGGCGTGATAAGTTTTATGGTATCCCCGGACGTTGGCTAC'),
    ('b', 'TGGTGCGAAAGCATCTCTTTTCGTGGCGTGATAGTTTTATGGTATCCCCGGAACGCTGGCTAC'),
    ('c', 'TCTGTGCGATATACATCTCTATCGTTGCGGTATGTTTTATGTGCATACCCCCACGCGCTGGCTAC'),
    ('d', 'ATGAGGCGCAAAATTCTCTTTCTCGTGCGCTGATTAAGTTTTATGTATCCCCGGACGTTGGCTAC'),
    ('e', 'TGGTGCGATATACATCTCTTTTCGTGCGTATGTTTTATGGTGATACCCCGGAACCGCTGGCTAC'),
    ('f', 'GGGGGAAAGCGATCCCTTATCGTGGCTGTGATAAGTTTTATCGGGTATCCGCCGGACGTTGGCTAC'),
    ('g', 'TGGTGCGATATACATCCTCTTTTCGTGCGTATGTTTTAGGTACACCGGATACGCCTGGCTTAC'),
    ]

genealogy_tree = GreedyTree(set_strings)
print(genealogy_tree.build_tree())

```

```

['a', 'd', 'f', 'e', 'c', 'b', 'g', None, None, None, None, None, None, None, None, None]

```

tests

```

In [17]: set_strings_test1 = [
    ('a', 'AAAAAA'),

```

```

        ('b', 'BAAAAA'),
        ('c', 'CAAAAA'),
        ('d', 'BDAALW'),
        ('e', 'BEAAMX'),
        ('f', 'CFAANY'),
        ('g', 'CGAAOZ'),
    ]

    set_strings_test2 = [
        ('d', 'BDAALW'),
        ('e', 'BEAAMX'),
        ('f', 'CFAANY'),
        ('g', 'CGAAOZ'),
        ('a', 'AAAAAA'),
        ('b', 'BAAAAA'),
        ('c', 'CAAAAA'),
    ]

    set_strings_test3 = [
        ('a', 'AAAAAA'),
        ('b', 'AAAAAA'),
        ('c', 'AAAAAA'),
        ('d', 'AAAAAA'),
        ('e', 'AAAAAA'),
        ('f', 'AAAAAA'),
        ('g', 'AAAAAA'),
    ]

    gene_tree_1 = GreedyTree(set_strings_test1)
    gene_tree_2 = GreedyTree(set_strings_test2)
    gene_tree_3 = GreedyTree(set_strings_test3)
    print(gene_tree_1.build_tree())
    print(gene_tree_2.build_tree())
    print(gene_tree_3.build_tree())

    assert gene_tree_1.build_tree() == gene_tree_2.build_tree()

```

```

['a', 'b', 'c', 'd', 'e', 'f', 'g', None, None, None, None, None, None, None, None]
['a', 'b', 'c', 'd', 'e', 'f', 'g', None, None, None, None, None, None, None, None]
['a', 'b', 'c', 'd', 'e', 'f', 'g', None, None, None, None, None, None, None, None]

```

Appendix 3b

```

In [19]: class DynamicTree(LCSMatrix):
    """
    A class that reconstructs the genetic tree from a dictionary of strings.
    This is found by recursively calling on the subtrees of the tree by greedy
    and optimizing the sum of each sub tree with the metric between the root
    and the subtrees.

    Parameters:
    -----
        set_strings : dictionary

```

```

        dictionary with name of string and string you wish to compute

Returns:
-----
relatability
tree in an array form

"""

def __init__(self, set_strings):
    super().__init__(set_strings)
    self.memo_trees = {}
    self.memo_perc = {}

def relatability_percent(self, lst_str1, idx1, lst_str2, idx2):
    """
    From some label of the dictionary it returns the index in the original
    dictionary

    Parameters
    -----
    lst_str1: list
        list of strings that we want to find the first index of

    idx1: int
        the index of the first string we want to find the relatability

    lst_str2: list
        list of strings that we want to find the second index of

    idx2: int
        the index of the first string we want to find the relatability

    Returns
    -----
    i: relatability
        probability that two strings are related to one another based on
        their LCS
    """
    key = (
        tuple(lst_str1[idx1]),
        tuple(lst_str2[idx2]),
    ) # extracts the strings themselves
    if (
        key in self.memo_perc
    ): # memoizes it so that we do not need to recompute the relatability
        return self.memo_perc[key]

    self.lena = len(lst_str1[idx1][1]) # length of string 1
    self.lenb = len(lst_str2[idx2][1]) # length of string 2

    my_lcs = LongestCommonSubsequences(lst_str1[idx1][1], lst_str2[idx2][1])
    len_lcs = (
        my_lcs.get_lcs_len()
    ) # gets the LCS length between the two strings

    len_smallest_string = min(
        self.lena, self.lenb
    ) # finds the minimum length between the two strings
    relatability = (
        len_lcs / len_smallest_string
    ) # calculates the probability that they are related by dividing

```

```

        self.memo_perc[
            key
        ] = relatability # adds it into the memo_perc dictionary with the
        return relatability

def generate_combinations(self, remaining_strings, half):
    """
    generates all possible combinations of half the length of the num

    Here is my own code for this function so you see that I can do th

        n = len(remaining_strings)
        combinations = []

        for i in range(1 << n):
            current = []
            for j in range(n):
                if i & (1 << j):
                    current.append(remaining_strings[j])

            if len(current) == half:
                combinations.append(current)

        return combinations

Parameters
-----
remaining_strings: list
                    list of strings that we want to generate combin

half: int
      k the length of the combination we want to make but for our

Returns
-----
i: relatability
   probability that two strings are related to one another based o
    """
    return [
        list(combo_half)
        for combo_half in combinations(remaining_strings, half)
    ] # generates the combinations using iter tools

def build_tree(self, strings): # class to builds the actual tree
    """
    method to build the actual tree by recursively builds the tree fo
    based on the sum of the recursive maximums of the left and right
    This follows a bottom up python implementation as it starst from

Parameters
-----
strings: list
         list of strings that we want to build a tree for

Returns
-----

```



```

result: The best root, maximum relatability totalled, and best le
"""
memo_key = tuple(
    sorted(strings)
) # sorts these strings because if it already has encountered th
if (
    memo_key in self.memo_trees
): # if the set of strings already exists then it just returns t
    return self.memo_trees[memo_key]

if (
    len(strings) == 1
): # sets up base case, the recursion works bottom up, however,
    result = (strings[0], 0, None, None)
    self.memo_trees[memo_key] = result
    return result

max_sum = -1 # defines max sum for the given tree
best_root = None # defines the best possible root
best_l_tr = None # defines best possible left sub tree combinati
best_r_tr = None # defines best possible right sub tree combinat

for i, root in enumerate(
    strings
): # we iterate through the dictionary with the index and the ro
    remaining_strings = (
        strings[:i] + strings[i + 1 :]
    ) # we have the remaining strings and remove the root so its
    l_combos = self.generate_combinations(
        remaining_strings, len(remaining_strings) // 2
    ) # generates all possible combinations for the left sub tree

    for l in l_combos: # for all possible left sub trees
        r = [
            str for str in remaining_strings if str not in l
        ] # it creates the right sub tree by taking the remainin

        (
            left_root,
            ls,
            ll,
            lr,
        ) = ( # then it recursively builds the sub trees for the
            self.build_tree(l) if l else (None, 0, None, None)
        )
        (
            right_root,
            rs,
            rl,
            rr,
        ) = ( # then it recursively builds the sub trees for the
            self.build_tree(r) if r else (None, 0, None, None)
        )

        l_relatability = ( # calculates the relatability from th
            self.relatability_percent(
                strings,
                i,
                remaining_strings,
                remaining_strings.index(left_root),

```

```

        )
        if left_root is not None
        else 0
    )
    r_relatability = ( # calculates the relatability from the
        self.relatability_percent(
            strings, i, r, r.index(right_root)
        )
        if right_root is not None
        else 0
    )

    total_sum = (
        ls + rs + l_relatability + r_relatability
    ) # calculates the total sum as the sum between in left and right

    if (
        total_sum > max_sum
    ): # if the total max for this is greater the general max
        max_sum = total_sum
        best_root = root # sets the root as the new best root
        best_l_tr = (
            left_root,
            ls,
            ll,
            lr,
        ) # defines the best left subtree based on the best
        best_r_tr = (
            right_root,
            rs,
            rl,
            rr,
        ) # defines the best right subtree based on the best

    result = (
        best_root,
        max_sum,
        best_l_tr,
        best_r_tr,
    ) # after going through all possible combinations it returns the
    self.memo_trees[
        memo_key
    ] = result # memoizes this result (so we dont need to recalculate)
    return result

def rebuild_tree(self):
    """
    This method just calls on the actual build tree to make it so that

    Parameters
    -----
    None

    Returns
    -----
    same as the previous class
    """
    root, relatability_sum, left_subtree, right_subtree = self.build_tree(
        self.set_strings
    )

```

```

        return root, relatability_sum, left_subtree, right_subtree

def print_tree(self, tree, lvl=1):
    """
    This just visually prints the tree so that you can see it

    Parameters
    -----
    tree: object
        the tree you want to visualize from this class

    lvl: int
        this is for spacing

    Returns
    -----
    None (prints the tree)
    """
    if tree is None: # once there is nothing left just terminate
        return
    root, sv, l_tree, r_tree = tree
    print(
        " " * lvl * 4 + f"root: {root[0]}, relatability: {sv}"
    ) # prints the root and the relatability indented based on the lvl
    if (
        l_tree
    ): # recursively calls on print tree for the left sub tree adds .
        self.print_tree(l_tree, lvl + 1)
    if (
        r_tree
    ): # recursively calls on print tree for the left sub tree adds .
        self.print_tree(r_tree, lvl + 1)

```

```

In [20]: set_strings = [
    ('a', 'ATGGTGCGAAAGCATCTCTTTTCGTGGCGTGATAAGTTTTATGGTATCCCCGGACGTTGGCTAC')
    ('b', 'TGGTGCGAAAGCATCTCTTTTCGTGGCGTATAGTTTTATGGTATCCCCGGAACGCTGGCTAC')
    ('c', 'TCTGTGCGATATACATCTCTATCGTTGCGGTATGTTTTATGTGCATCACCCACGCGCTGGCTAC')
    ('d', 'ATGAGGCGCAAAATTCTCTTTCTCGTGGCGCTGATTAAGTTTTATGTATCCCCGGACGTTGGCTAC')
    ('e', 'TGGTGCGATATACATCTCTTTTCGTGCGTATGTTTTATGGTGATCACCCGGAACCGCTGGCTAC')
    ('f', 'GGGGGAAAGCGATCCCTTATCGTGGCTGTGATAAGTTTTATCGGGTATCCGCCGGACGTTGGCTAC')
    ('g', 'TGGTGCGATATACATCCTCTTTTCGTGCGTATGTTTTAGGTACACCGGATACGCCTGGCTTAC')
]

builder = DynamicTree(set_strings)
tree = builder.rebuild_tree()
builder.print_tree(tree)

```

```

root: b, relatability: 5.638970350404313
  root: a, relatability: 1.8981132075471698
    root: d, relatability: 0
    root: f, relatability: 0
  root: e, relatability: 1.8928571428571428
    root: c, relatability: 0
    root: g, relatability: 0

```

tests

```

In [22]: set_strings_test1 = [
    ('a', 'AAAAAA'),

```

```

        ('b', 'BAAAAA'),
        ('c', 'CAAAAA'),
        ('d', 'BDAALW'),
        ('e', 'BEAAMX'),
        ('f', 'CFAANY'),
        ('g', 'CGAAOZ'),
    ]

    set_strings_test2 = [
        ('b', 'BAAAAA'),
        ('a', 'AAAAAA'),
        ('c', 'CAAAAA'),
        ('d', 'BDAALW'),
        ('e', 'BEAAMX'),
        ('f', 'CFAANY'),
        ('g', 'CGAAOZ'),
    ]

    set_strings_test3 = [
        ('a', 'AAAAAA'),
        ('b', 'AAAAAA'),
        ('c', 'AAAAAA'),
        ('d', 'AAAAAA'),
        ('e', 'AAAAAA'),
        ('f', 'AAAAAA'),
        ('g', 'AAAAAA'),
    ]

    gene_tree_1 = DynamicTree(set_strings_test1)
    gene_tree_2 = DynamicTree(set_strings_test2)
    gene_tree_3 = DynamicTree(set_strings_test3)

    tree_1 = gene_tree_1.rebuild_tree()
    tree_2 = gene_tree_2.rebuild_tree()
    tree_3 = gene_tree_3.rebuild_tree()

    print("Tree 1:")
    gene_tree_1.print_tree(tree_1)

    print("\nTree 2:")
    gene_tree_2.print_tree(tree_2)

    print("\nTree 3:")
    gene_tree_3.print_tree(tree_3)

    assert tree_1 == tree_2, "Tree 1 and Tree 2 should be identical for the s

```

Tree 1:

```

    root: a, relatability: 3.666666666666667
        root: b, relatability: 1.0
            root: d, relatability: 0
            root: e, relatability: 0
        root: c, relatability: 1.0
            root: f, relatability: 0
            root: g, relatability: 0

```

Tree 2:

```

    root: a, relatability: 3.666666666666667
        root: b, relatability: 1.0
            root: d, relatability: 0

```

```

        root: e, relatability: 0
    root: c, relatability: 1.0
        root: f, relatability: 0
        root: g, relatability: 0

```

Tree 3:

```

    root: a, relatability: 6.0
        root: b, relatability: 2.0
            root: c, relatability: 0
            root: d, relatability: 0
        root: e, relatability: 2.0
            root: f, relatability: 0
            root: g, relatability: 0

```

Appendix 5

```

In [24]: # defining gene characters to create random gene strings
genes = ["A", "T", "G", "C"]

def generate_random_strings(num_strings, string_length):
    """
    generating lists of random strings using the random library dependant

    Parameters
    -----
    num_strings: int
        number of strings in the list

    string_length: int
        the length of each string

    Returns
    -----
    list of strings
    """
    return [
        (random.choice(genes), "").join(random.choices(genes, k=string_length))
        for i in range(num_strings)
    ]

def test_time_complexity_dynamic(num_strings, string_length):
    """
    calculates time complexity of dynamic approach

    Parameters
    -----
    num_strings: int
        number of strings in the list

    string_length: int
        the length of each string

    Returns
    -----
    elapsed_time: float
        amount of time took to build the tree
    """

```

```

    set_strings = generate_random_strings(
        num_strings, string_length
    ) # randomly generates a set of strings
    dynamic_tree = DynamicTree(
        set_strings
    ) # creates instance of class for that set of randomly generated str

    start_time = time.time() # sets start time
    dynamic_tree.rebuild_tree() # runs the build tree dynamic programming
    end_time = time.time() # stops time when done

    elapsed_time = end_time - start_time # elapsed time is start minus e
    return elapsed_time

times_dynamic = [] # list storing all elapsed times of dynamic

string_lengths_dynamic = (
    []
) # list for strings lengths that we wish to plot when we observing scal

for i in range(
    1, 50
): # runs a for loop to add the lengths to the list from 10 to 500 (incr
    string_lengths_dynamic.append(10 * i)

for (
    length
) in (
    string_lengths_dynamic
): # for every length of string it iterates and finds the elapsed time t
    num_strings = 7 # keeps number of strings constant
    elapsed_time = test_time_complexity_dynamic(
        num_strings=num_strings, string_length=length
    )
    times_dynamic.append(elapsed_time)

def test_time_complexity_greedy(num_strings, string_length):
    """
    calculates time complexity of greedy approach

    Parameters
    -----
    num_strings: int
        number of strings in the list

    string_length: int
        the length of each string

    Returns
    -----
    elapsed_time: float
        amount of time took to build the tree
    """
    set_strings = generate_random_strings(
        num_strings, string_length
    ) # randomly generates a set of strings
    greedy_tree_builder = GreedyTree(
        set_strings

```

```

    ) # creates instance of class for that set of randomly generated str.

    start_time = time.time() # sets start time
    greedy_tree_builder.build_tree() # runs the build tree dynamic progr
    end_time = time.time() # stops time when done

    elapsed_time = end_time - start_time # elapsed time is start minus e
    return elapsed_time

times_greedy = [] # list storing all elapsed times of greedy

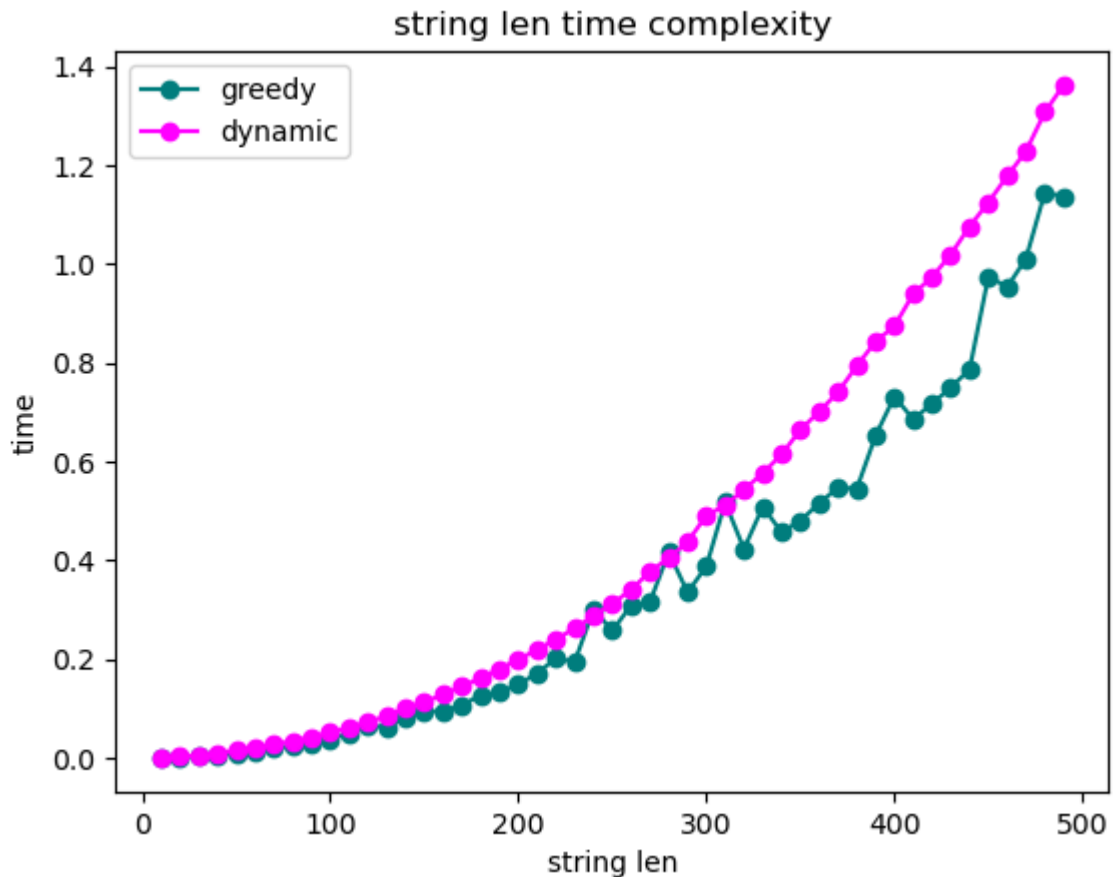
string_lengths_greedy = (
    []
) # list for strings lengths that we wish to plot when we observing scal

for i in range(
    1, 50
): # runs a for loop to add the lengths to the list from 10 to 500 (incr
    string_lengths_greedy.append(10 * i)

for (
    length
) in (
    string_lengths_greedy
): # for every length of string it iterates and finds the elapsed time t
    num_strings = 7 # keeps number of strings constant
    elapsed_time = test_time_complexity_greedy(
        num_strings=num_strings, string_length=length
    )
    times_greedy.append(elapsed_time)

plt.plot(
    string_lengths_greedy,
    times_greedy,
    marker="o",
    linestyle="-",
    color="teal",
    label="greedy",
) # plots greedy
plt.plot(
    string_lengths_dynamic,
    times_dynamic,
    marker="o",
    linestyle="-",
    color="magenta",
    label="dynamic",
) # plots dynamic
plt.xlabel("string len") # adds x label
plt.ylabel("time") # adds y label
plt.title("string len time complexity") # adds title
plt.legend() # adds a legend
plt.show() # shows the plot

```



```
In [25]: # same as above but keeps the stringlength constant and sees how the number of
times_dynamic = []

num_strings_dynamic = []
for i in range(1, 17):
    num_strings_dynamic.append(i)

for num in num_strings_dynamic:
    string_length = 3
    elapsed_time = test_time_complexity_dynamic(
        num_strings=num, string_length=string_length
    )
    times_dynamic.append(elapsed_time)

times_greedy = []
num_strings_greedy = []
for i in range(1, 17):
    num_strings_greedy.append(i)

for num in num_strings_greedy:
    string_length = 3
    elapsed_time = test_time_complexity_greedy(
        num_strings=num, string_length=string_length
    )
    times_greedy.append(elapsed_time)

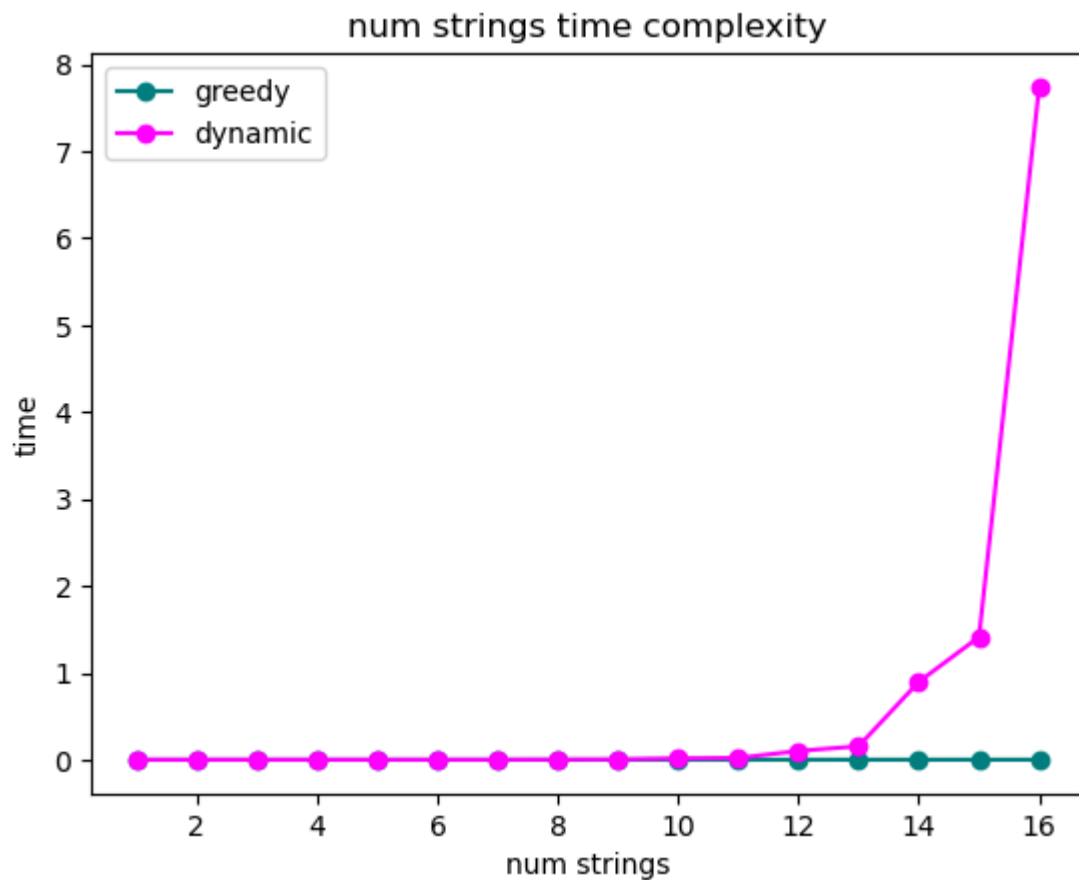
plt.plot(
    num_strings_greedy,
    times_greedy,
    marker="o",
    linestyle="--",
    color="teal",
```



```

        label="greedy",
    )
    plt.plot(
        num_strings_dynamic,
        times_dynamic,
        marker="o",
        linestyle="--",
        color="magenta",
        label="dynamic",
    )
    plt.xlabel("num strings")
    plt.ylabel("time")
    plt.title("num strings time complexity")
    plt.legend()
    plt.show()

```



```

In [26]: # just plots the greedy approach for better visualization of individual s
times_greedy = []
num_strings_greedy = []
for i in range(1, 50):
    num_strings_greedy.append(i * 10)

for num in num_strings_greedy:
    string_length = 3
    elapsed_time = test_time_complexity_greedy(
        num_strings=num, string_length=string_length
    )
    times_greedy.append(elapsed_time)

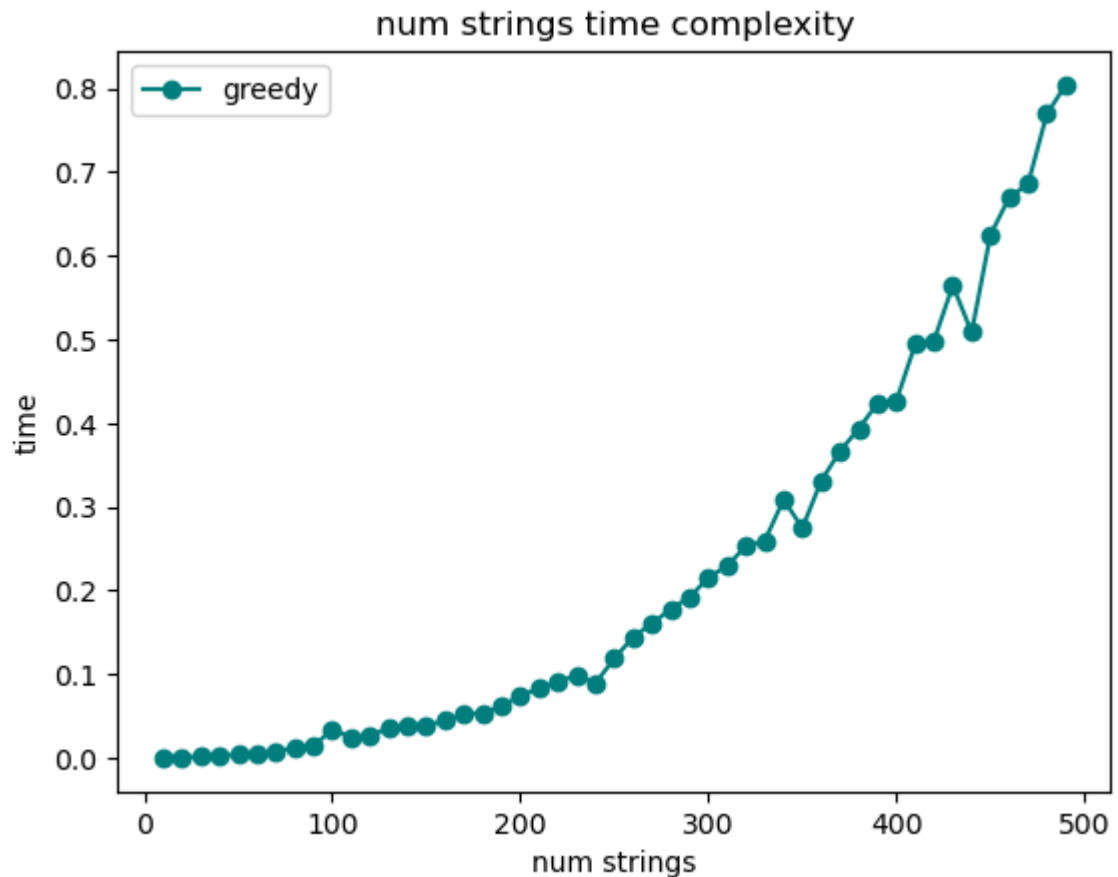
plt.plot(
    num_strings_greedy,
    times_greedy,
    marker="o",

```

```

linestyle="--",
color="teal",
label="greedy",
)
plt.xlabel("num strings")
plt.ylabel("time")
plt.title("num strings time complexity")
plt.legend()
plt.show()

```



Appendix 6

```

In [28]: from sympy import symbols, Eq, solve

def prob_mutants(par_lst, chld_lst, lcs_lst):
    """
    A function that calculates the probability of insertion, deletion, mu

    Parameters:
    -----
    par_lst: list (of ints)
              list of parents lengths that then are mutated into child

    chld_lst: list (of ints)
              list of chlds lengths that were mutated from parents a

    lcs_lst: list (of ints)
             list of lcs of the parents and chlds

    Returns:

```

```

-----
avg_i: float
    average probability of insertion

avg_m: float
    average probability of mutation

avg_d: float
    average probability of deletion

avg_n: float
    average probability of doing nothing to a character
"""
iv = [] # initialize list to hold insertion probabilities
mv = [] # initialize list to hold mutation probabilities
dv = [] # initialize list to hold deletions probabilities
nv = [] # initialize list to hold nothing probabilities

for idx in range(
    len(par_lst)
): # cycles through the range of the lists to iterate through all th
    par = par_lst[
        idx
    ] # sets the parent as the int with the index in the parent's li
    chld = chld_lst[
        idx
    ] # sets the child as int with the index in the children's list
    lcs = lcs_lst[
        idx
    ] # sets the lcs as int with the index in the lcs's list

    n, i, d, m = symbols("n i d m") # defines the variables

    eq1 = Eq(n, 1 - (i + d + m)) # sum of all probabilities are equa
    eq2 = Eq(
        i, (lcs - par * n) / par
    ) # the lcs is the number of characters that had an insertion an
    eq3 = Eq(
        d, (par + par * i - chld) / par
    ) # the child is the same as the parent plus the number of inser
    eq4 = Eq(
        m, (i * d) * 2
    ) # mutation is same as adding an insertion then deleting the ne.

    solution = solve(
        (eq1, eq2, eq3, eq4), (n, i, d, m)
    ) # solves the equations using the library

    n_val = solution[0][
        0
    ] # gets the first solution which is the positive one for n
    i_val = solution[0][1] # does the same for i,d,m
    d_val = solution[0][2]
    m_val = solution[0][3]

    iv.append(
        i_val
    ) # adds the i solution to a list to later find the average of
    mv.append(m_val) # does the same for m,d,n
    dv.append(d_val)

```

```

        nv.append(n_val)

    avg_i = sum(iv) / len(iv)  # finds the average of all i in the list
    avg_m = sum(mv) / len(mv)  # does the same for m,d,n
    avg_d = sum(dv) / len(dv)
    avg_n = sum(nv) / len(nv)

    return (
        round(float(avg_i), 4),
        round(float(avg_m), 4),
        round(float(avg_d), 4),
        round(float(avg_n), 4),
    )  # returns these as floats

par_lst = [
    250,
    250,
    265,
    265,
    252,
    252,
]
]  # sets the parents list, childs list, and lcs lst based on the lcs mat.
chld_lst = [265, 252, 282, 280, 257, 269]
lcs_lst = [235, 227, 252, 251, 234, 243]

avg_i, avg_m, avg_d, avg_n = prob_mutants(par_lst, chld_lst, lcs_lst)
print(f"i: {avg_i}, m: {avg_m}, d: {avg_d}, n: {avg_n}")

```

i: 0.0966, m: 0.0096, d: 0.0506, n: 0.8432

In []: